CUSTOMER RETENTION INTELLIGENCE SYSTEM

Progressive Lab Guide — Student Edition

Dataset:

Kaggle — Telecom Churn Dataset (suraj520)

kaggle.com/datasets/suraj520/telecom-churn-dataset

Each lab includes:

Activity Guide — stepwise student activities, concepts, expected output, connections

Training Phases:

Phase 1 — Python Core (CP1–CP5)
Phase 2 — Database Layer (SQL1–SQL6 + ORM)
Phase 3 — FastAPI Backend (API1–API6)
Phase 4 — React UI (RE1–RE5)
Phase 5 — Data Engineering (DE1–DE8, incl. PySpark)
Phase 6 — Machine Learning (ML1–ML5)

IBM/Telco Churn Dataset · 7,043 customers · 21 columns · No size constraints on 16 GB laptops

DATASET REFERENCE — READ THIS FIRST

This dataset is the IBM/Telco Customer Churn dataset — one of the most widely used churn datasets in machine learning. It represents a customer-state snapshot: one row per customer, capturing their profile, service subscriptions, billing, and whether they churned.

DATASET

Source: Kaggle — kaggle.com/datasets/suraj520/telecom-churn-dataset
Size: 7,043 rows · 21 columns · ~1 MB CSV — NO memory concerns on 16 GB laptops
No subsetting needed: load the full file in every lab
Churn distribution: ~73.5% No (active), ~26.5% Yes (churned) — mild class imbalance
This is a customer-STATE dataset, not an event stream. One row = one customer snapshot.

Setup — One-Time

No preprocessing required. Simply download the CSV from Kaggle and rename it to customer_churn.csv. Distribute to students via USB or LMS before Day 1.

```
# Verify the file is correct after download:
import pandas as pd
df = pd.read_csv("customer_churn.csv")
assert df.shape == (7043, 21), f"Unexpected shape: {df.shape}"
assert "customerID" in df.columns, "Missing customerID column"
assert "Churn" in df.columns, "Missing Churn column"
print("Dataset OK:", df.shape)
```

Column Schema — All 21 Columns

Column	Type	Description
customerID	object	Unique customer identifier. DROP before ML — not a feature.
gender	object	Male / Female
SeniorCitizen	int64	0 or 1 (already numeric — no encoding needed)
Partner	object	Yes / No — encode to 1/0 in CP2
Dependents	object	Yes / No — encode to 1/0 in CP2
tenure	int64	Months with the company (0–72)
PhoneService	object	Yes / No
MultipleLines	object	Yes / No / No phone service
InternetService	object	DSL / Fiber optic / No
OnlineSecurity	object	Yes / No / No internet service
OnlineBackup	object	Yes / No / No internet service
DeviceProtection	object	Yes / No / No internet service

Column	Type	Description
TechSupport	object	Yes / No / No internet service
StreamingTV	object	Yes / No / No internet service
StreamingMovies	object	Yes / No / No internet service
Contract	object	Month-to-month / One year / Two year
PaperlessBilling	object	Yes / No
PaymentMethod	object	4 values — includes "automatic" variants for auto_pay_flag
MonthlyCharges	float64	Monthly bill amount. Always numeric.
TotalCharges	object	STORED AS STRING. Has ~11 blank values for tenure=0 customers. Fix in CP2.
Churn	object	Target: Yes / No. Encode to 1/0 in CP2. This is y in all ML labs.

CRITICAL

TotalCharges is stored as object (string), not float. It has ~11 blank " " values (space, not NaN).
Fix: pd.to_numeric(df["TotalCharges"], errors="coerce") converts blanks to NaN.
These 11 blanks are customers with tenure=0 — they have no total charges yet. Fill with MonthlyCharges.
customerID MUST be dropped before any ML training. It is an identifier, not a feature.
Binary Yes/No columns must be encoded to 1/0 before ML. Done in CP2.

TOC Cross-Reference

This lab set covers the following TOC sections. Two items require specific attention:

TOC Section	Coverage in This Lab Set
Python Core — OOP, CSV/JSON, error handling, logging	CP1–CP5 — CustomerCleaner class, pipeline script
Database Layer — SQL essentials, DB connectivity, ORM (SQLAlchemy)	SQL1–SQL6 — staging, curated tables, SQLAlchemy models in SQL3
FastAPI — routing, Pydantic, error handling	API1–API6 — full backend including validation and auth stub
React — API consumer, useState, useEffect, fetch	RE1–RE5 — all React patterns covered
Data Engineering — ETL, pipeline architecture, orchestration	DE1–DE7 — batch customer intelligence pipeline with Airflow
Machine Learning — feature engineering, model training, evaluation	ML1–ML5 — churn classification, interpretability, batch scoring

TOC Section	Coverage in This Lab Set
PySpark — DataFrame API, transformations, Parquet	DE5 — PySpark lab inside Phase 5 DE track (reads customer data, churn aggregations, Parquet write)
MongoDB/NoSQL — NOT covered in this lab set	⚠ Covered in separate MongoDB module or use network dataset track
Power BI — NOT covered in this lab set	⚠ Suggested addition: connect Power BI to SQL3 customer tables

PHASE 1

PYTHON CORE

Understand and preprocess telecom customer data

LAB CP1 — Profile the Customer Dataset

Goal: Understand structure and quality before any transformation

Prereq: customer_churn.csv available

▶ ACTIVITY GUIDE

SCENARIO	• You have just received a raw customer extract from the CRM team. Before anyone touches this data, your job is to profile it — undertypes, quality, and any anomalies. Producits e a written summary of what you found.stand shape,
STUDENT ACTIVITIES	1. Load customer_churn.csv using pandas and print its shape (rows, columns) 2. Print all column names and their data types 3. Count null values per column — calculate the null percentage for each 4. For each key categorical column (gender, Partner, Dependents, PhoneService, InternetService, Contract, PaymentMethod, Churn) print unique values and value counts 5. Count churned vs non-churned customers — calculate the churn rate as a percentage 6. Inspect the TotalCharges column specifically: print its dtype, then try pd.to_numeric() with errors="coerce" and count how many NaN values appear 7. Print min/max/mean for tenure, MonthlyCharges, TotalCharges. 8. Write a short profiling summary: 5 bullet points of things the data team needs to know before using this data
CONCEPTS	• pandas read_csv, shape, dtypes, isnull(), value_counts() • pd.to_numeric() with errors="coerce" — converting mixed-type columns • normalize=True in value_counts() — returns proportions not counts • Data profiling mindset: never transform before you understand • Class imbalance: churn datasets are typically ~73% No / ~27% Yes — note for ML later • customerID is an identifier, not a feature — must be excluded from ML
OUTPUT	• Printed shape: (7043, 21) • Null count table — TotalCharges will have ~11 blanks encoded as spaces • Churn distribution: ~26.5% churned (Yes), ~73.5% not churned (No) • Written 5-bullet profiling summary per student
CONNECTS TO	**CP2:** TotalCharges blank detection here drives the fix_total_charges() method **CP3:** Churn distribution insight motivates the business analysis labs **ML1:** customerID identified here as a column to drop before modelling

LAB CP2 — Build a Reusable Cleaning Utility

Goal: Convert raw data into a cleaned DataFrame that all downstream labs can trust

Prereq: CP1 completed

▶ ACTIVITY GUIDE

SCENARIO	• The profiling showed several data quality issues: TotalCharges is stored as a string, Yes/No fields need encoding, and column names are inconsistent casing. Build a reusable CustomerCleaner class that fixes all of this reliably — it will be imported by every downstream lab.
STUDENT ACTIVITIES	9. Create customer_cleaner.py with a CustomerCleaner class that accepts a DataFrame in __init__ 10. Implement standardize_column_names() — convert all column names to lowercase snake_case (MonthlyCharges → monthly_charges) 11. Implement fix_total_charges() — replace blank strings with NaN, then convert to float using pd.to_numeric(errors="coerce"); log how many rows were affected 12. Implement normalize_binary_columns() — map Yes→1 and No→0 for: partner, dependents, phone_service, paperless_billing, churn 13. Implement handle_nulls() — for total_charges nulls, fill with monthly_charges value (justified: new customers with tenure=0 have no total charges yet) 14. Implement clean() — calls all methods in sequence and returns the cleaned DataFrame 15. Add Python logging statements at each step (use logging.INFO, not print()) 16. Add try/except around fix_total_charges() and log any unexpected errors 17. Test: instantiate CustomerCleaner, call .clean(), print shape and dtypes before/after
CONCEPTS	• OOP: class, __init__, instance methods, method chaining with return self • df.copy() in __init__ — why mutating the original is dangerous • pd.to_numeric(errors="coerce") — coerce turns unparseable values to NaN • str.lower() + str.replace() for column name normalization • Python logging module: logging.getLogger(__name__), INFO vs WARNING vs ERROR • try/except for defensive data handling: catch unexpected type errors gracefully • Filling nulls with monthly_charges — justified by business context (tenure=0 customers)
OUTPUT	• customer_cleaner.py importable from any other script • Cleaned DataFrame: TotalCharges is float64, binary cols are int (0/1), column names are snake_case • Log output showing each transformation step with row counts • Zero crashes when clean() is called on the raw CSV
CONNECTS TO	**CP3:** clean() output is the DataFrame used for business analysis groupbys **CP5:** CustomerCleaner is imported and called in the pipeline script **SQL2:** Cleaned column names (snake_case) must match the staging table schema **DE3:** The cleaning stage in the pipeline reuses this same class

LAB CP3 — Generate Business Insights

Goal: Use Python to answer 5 telecom retention questions

Prereq: CP2 — CustomerCleaner output

▶ ACTIVITY GUIDE

SCENARIO	• The retention team wants to know which customer segments are most at risk of churning. Use the cleaned data to answer 5 business questions with groupby analysis, then write a short "findings" paragraph that a non-technical manager could read.
STUDENT ACTIVITIES	18. Load and clean the data using CustomerCleaner 19. Compute churn rate by contract type — sort descending. Which contract has the highest churn? 20. Compute churn rate by internet service type 21. Compute churn rate by payment method 22. Create a tenure_bucket column using pd.cut() with bins [0,12,24,48,72] and labels ["0-12","13-24","25-48","49-72"]. Compute churn rate per bucket. 23. Compute average monthly_charges for churned vs non-churned customers 24. Find the combination (contract + internet_service) with the highest churn rate using groupby on two columns 25. Write a 3-sentence business summary: which segments are highest risk and what that means for the retention team
CONCEPTS	• pd.cut() — binning continuous variables into categorical buckets • groupby().mean() on a 0/1 encoded column gives churn rate directly • Multi-column groupby — groupby(["col1","col2"]) • sort_values(ascending=False) — rank segments by risk • Business interpretation: translating statistical outputs into plain language • Expected finding: month-to-month + fiber optic = highest churn risk segment
OUTPUT	• Churn rate table per contract (month-to-month will be highest, ~43%) • Churn rate table per tenure bucket (0-12 months will be highest) • Churn rate per payment method (electronic check tends to be highest) • Written 3-sentence manager summary per student
🔗 CONNECTS TO	**SQL4:** These exact business questions are re-answered in SQL4 using SQL GROUP BY **CP4:** tenure_bucket created here becomes a formal engineered feature in CP4 **API2:** GET /churn/summary endpoint returns these same aggregations from the database **ML3:** Segment-level churn rates guide feature importance interpretation

LAB CP4 — Build Engineered Features

Goal: Prepare ML-ready features — performance depends on feature design

Prereq: CP2 output

▶ ACTIVITY GUIDE

SCENARIO	• Raw columns alone do not make a good ML model. You need to engineer features that capture customer behaviour patterns. Create 6 derived features and save the result as customer_features.csv — this file becomes the ML training input.
STUDENT ACTIVITIES	26. Start from the cleaned DataFrame (CustomerCleaner output) 27. Feature 1 — tenure_bucket: pd.cut() with bins [0,12,24,48,72] as in CP3 28. Feature 2 — high_charge_flag: 1 if monthly_charges > median monthly_charges, else 0 29. Feature 3 — service_count: count how many of these 6 service columns equal 1: online_security, online_backup, device_protection, tech_support, streaming_tv, streaming_movies 30. Feature 4 — is_long_term_customer: 1 if tenure >= 24, else 0 31. Feature 5 — has_streaming_bundle: 1 if both streaming_tv=1 AND streaming_movies=1, else 0 32. Feature 6 — auto_pay_flag: 1 if payment_method contains "automatic", else 0 (use str.contains()) 33. Print the correlation of each new feature with the churn column 34. Save to customer_features.csv — include all original cleaned columns plus the 6 new ones
CONCEPTS	• Feature engineering: creating model inputs from raw business data • .median() as a threshold — robust to outliers vs mean • .astype(int) — converts boolean True/False to 1/0 • df[cols].sum(axis=1) — row-wise sum across multiple columns • str.contains() — partial string match for categorical encoding • .corrwith() — correlation of multiple columns with a target • ML performance depends heavily on feature design, not only on algorithms
OUTPUT	• customer_features.csv saved with all original columns + 6 new feature columns • Correlation table: high_charge_flag and short tenure likely show positive correlation with churn • service_count likely shows negative correlation with churn (more services = less likely to leave)
CONNECTS TO	**CP5:** build_features() function in the pipeline calls this same logic **SQL5:** customer_ml_features SQL table mirrors the columns in this CSV **ML1:** This file is the direct input to ML problem definition **ML2:** Feature columns defined here are the X matrix for model training

LAB CP5 — Build End-to-End Preprocessing Pipeline

Goal: Package everything into one executable job — bridge to SQL and DE

Prereq: CP1–CP4 all completed

▶ ACTIVITY GUIDE

SCENARIO	• Individual scripts are not enough for production. Package everything — loading, cleaning, feature engineering, and saving — into a single executable pipeline script that can be run daily with one command.
STUDENT ACTIVITIES	35. Create customer_pipeline.py 36. Import CustomerCleaner from customer_cleaner.py and build_features() logic 37. Define load_data(filepath) — reads CSV and logs row count 38. Define build_features(df) — runs all 6 feature engineering steps from CP4 and returns the enriched DataFrame 39. Define save_outputs(clean_df, feature_df, output_dir) — saves both files with timestamps in filenames 40. Define main() — calls load → clean → build_features → save in sequence 41. Add logging throughout: start time, step completion, row counts, output file paths 42. Wrap each step in try/except — if any step fails, log the error and raise; do not silently continue 43. Add a simple data quality check before saving: assert no nulls in monthly_charges, assert churn column has only 0/1 values 44. Run the script end-to-end: python customer_pipeline.py
CONCEPTS	• Script architecture: load → clean → transform → save as a linear, testable flow • try/except/raise — catch, log, and re-raise to let the caller know something failed • assert statements — lightweight data quality gates inside the pipeline • Timestamped output filenames: datetime.now().strftime("%Y%m%d") prevents overwriting • Separation of concerns: each function does one thing • if __name__ == "__main__" — allows import without executing
OUTPUT	• customer_pipeline.py runnable from command line • Two output files: cleaned_customer_YYYYMMDD.csv and customer_features_YYYYMMDD.csv • Clean log output showing all steps with timestamps • Pipeline raises an error (not silently fails) if data quality check fails
CONNECTS TO	**SQL2:** The cleaned CSV output is what gets loaded into the staging table **DE2:** This script becomes the processing step in the DE ingestion pipeline **DE3:** CustomerCleaner is reused directly in the DE cleaning stage

ML2: customer_features CSV is the direct input to the ML training script

PHASE 2

DATABASE LAYER

SQL + ORM — Model and query telecom customer intelligence

This phase covers the TOC Database Layer module: SQL essentials, DB connectivity, CRUD, SQLAlchemy ORM. SQL1–SQL5 use raw SQL; SQL3 introduces SQLAlchemy models. Students should have MySQL, Postgres, or SQLite installed.

TOC LINK	TOC section: "Database Layer (SQL + ORM)" — SQL essentials, DB connectivity, ORM using SQLAlchemy SQL3 is where SQLAlchemy is introduced — students define model classes and use sessions. SQL6 sets up the rule-based foundation that ML later improves on.

LAB SQL1 — Design the Relational Schema

Goal: Decompose flat CSV into logical entities before writing any SQL

Prereq: CP1 profiling complete — students know all 21 columns

▶ ACTIVITY GUIDE

SCENARIO	• The customer data currently lives in a flat CSV. Your task is to design a proper relational schema before writing any SQL — decomposing the 21 columns into logical entities. Design on paper first, then validate it with the trainer before creating any tables.
STUDENT ACTIVITIES	45. List all 21 columns from the dataset and categorise each as: stable identity, service subscription, billing/charges, or churn outcome 46. Define the entities: customers (identity), contracts (subscription terms), services (what they subscribe to), billing (charges), customer_status (churn) 47. Draw the entity-relationship diagram: which tables have foreign keys to which? 48. Define the primary key for each table (customer_id is the natural key throughout) 49. Identify which columns are dimension-like (contract type, payment method, internet service) vs measure-like (tenure, monthly_charges, total_charges) 50. Write a CREATE TABLE DDL statement for each of the 4–5 tables 51. Discuss with trainer: should this be a normalised OLTP schema or a denormalised analytics schema? Why?
CONCEPTS	• Entity decomposition: separating identity, subscription, billing, and outcome

	• Primary key vs foreign key — referential integrity • Natural key (customer_id as string) vs surrogate key (serial integer) • Normalised vs denormalised: normalised = less redundancy; denormalised = faster analytics reads • Staging table concept: one flat table matching the CSV structure, used as a landing zone • Dimension table: lookup values like contract type, payment method
OUTPUT	• Entity-relationship diagram (whiteboard or draw.io export) • Written column categorisation table • CREATE TABLE DDL for 4–5 tables • Verbal answer: "We chose [normalised/denormalised] because..."
CONNECTS TO	**SQL2:** Staging table is created to match raw CSV before any transformation **SQL3:** Curated tables built here are populated in SQL3 **API1:** FastAPI queries these exact tables to return customer profiles **DE1:** Pipeline architecture in DE1 maps to these storage layers

LAB SQL2 — Load Raw Data into Staging

Goal: Introduce the staging layer — raw data preserved exactly as received

Prereq: SQL1 schema designed

▶ ACTIVITY GUIDE

SCENARIO	• Before transforming anything, load the raw CSV into a staging table that mirrors the source format exactly. Staging is the safety net — if transformation goes wrong, the raw data is still intact.
STUDENT ACTIVITIES	52. Create a staging table stg_customer_raw with columns matching the raw CSV exactly (TotalCharges as VARCHAR, not FLOAT) 53. Load the CSV into the staging table using either: Python + SQLAlchemy to_sql(), or a MySQL LOAD DATA INFILE command 54. Verify row count: SELECT COUNT(*) FROM stg_customer_raw — should be 7043 55. Inspect TotalCharges in the staging table: SELECT TotalCharges FROM stg_customer_raw WHERE TotalCharges = "" LIMIT 5 56. Inspect data types: confirm TotalCharges is stored as text in staging (not yet converted) 57. Record the load time and row count in a simple ingestion_log table 58. Run a basic validation: count distinct customerIDs — should equal row count (no duplicates)
ONCEPTS	• Staging layer: raw data preserved as-is, no transformation yet • VARCHAR for TotalCharges in staging — matches the source reality • to_sql() with if_exists="replace" vs "append" — know when to use each • Row count verification: always check after loading

	• ingestion_log table: audit trail for every load (filename, row count, timestamp, status) • Duplicate detection: COUNT(*) vs COUNT(DISTINCT key)
OUTPUT	• stg_customer_raw table populated with 7043 rows • Row count verified: SELECT COUNT(*) = 7043 • Blank TotalCharges visible in staging (not yet cleaned) • ingestion_log entry created with load timestamp and row count
CONNECTS TO	**SQL3:** Curated tables are built by SELECT-INSERT from this staging table **CP2:** The TotalCharges blank issue confirmed here matches what CustomerCleaner fixes **DE2:** This manual loading process is automated in the DE ingestion pipeline

LAB SQL3 — Build Curated Tables + SQLAlchemy ORM

Goal: Transform staging into trusted tables; introduce SQLAlchemy models

Prereq: SQL2 staging loaded

▶ ACTIVITY GUIDE

SCENARIO	• The staging table is raw and messy. Now build the curated tables — clean, typed, and structured — that the rest of the system will actually use. Transform staging data into the schema you designed in SQL1.
STUDENT ACTIVITIES	59. Create dim_contract with rows for each unique contract type (Month-to-month, One year, Two year) 60. Create dim_payment with rows for each payment method 61. Create the customers table: INSERT from staging, standardising gender, casting senior_citizen, binary-encoding partner/dependents 62. Create fact_customer_account: INSERT tenure, charges, churn — cast TotalCharges with NULLIF to handle blanks, join to dim tables for foreign keys 63. Verify counts: each curated table row count should match expectations 64. Run a JOIN query across customers + fact_customer_account + dim_contract to confirm the relationships work 65. Introduce SQLAlchemy: create Python model classes for at least two of these tables (Customer, Contract) 66. Use a SQLAlchemy session to query one customer by customerID and print the result
CONCEPTS	• INSERT ... SELECT — transforming while loading • NULLIF(col, "") — convert blank string to NULL in SQL • CASE WHEN ... THEN — conditional encoding in SQL • SQLAlchemy: Base, Column, String, Integer, ForeignKey, relationship • SQLAlchemy Session: session.query(Customer).filter_by(customer_id="...").first() • The difference between staging (raw) and curated (trusted) tables

OUTPUT	• dim_contract: 3 rows (Month-to-month, One year, Two year) • dim_payment: 4 rows (one per payment method) • customers: 7043 rows with binary-encoded partner/dependents • fact_customer_account: 7043 rows with NULL for ~11 total_charges blanks • SQLAlchemy model query returning a Customer object successfully
CONNECTS TO	**SQL4:** Business analytics queries join these curated tables **SQL5:** ML feature table is built from fact_customer_account **API1:** FastAPI Customer Profile endpoint queries the customers + fact tables **API6:** SQLAlchemy ORM introduced here is used throughout all API labs

LAB SQL4 — Write Business Analytics Queries

Goal: Translate 5 retention questions into SQL — verify against CP3 pandas results

Prereq: SQL3 curated tables populated

▶ ACTIVITY GUIDE

SCENARIO	• The retention team has 5 business questions. Your job is to answer each one with a SQL query — no Python, no pandas. This demonstrates that well-structured SQL is often the fastest path from question to answer.
STUDENT ACTIVITIES	67. Query 1 — Churn rate by contract: SELECT contract_type, AVG(churn) AS churn_rate FROM fact_customer_account JOIN dim_contract GROUP BY contract_type ORDER BY churn_rate DESC 68. Query 2 — Churn rate by payment method: similar pattern, join with dim_payment 69. Query 3 — Average monthly charges by churn status: GROUP BY churn, return avg(monthly_charges) 70. Query 4 — Top risky segments: customers with month-to-month contract AND tenure < 12 AND monthly_charges > average — count and list 71. Query 5 — High-value churned customers: churned customers with total_charges > 2000 — how many and what is their average tenure? 72. For each query: first state the business question in plain English, then write the SQL, then interpret the result in one sentence 73. Bonus: write a single query that returns contract type, avg monthly charges, avg tenure, and churn rate together — sorted by churn rate
CONCEPTS	• GROUP BY + AVG on a 0/1 column = churn rate directly • Multi-table JOIN: fact + dimension • Subquery in WHERE clause: compare against a computed average • ORDER BY churn_rate DESC — ranking segments by risk • Translating business questions into filter + group + aggregate patterns • Expected result: month-to-month churn rate ~43%; two-year ~3%
OUTPUT	• Five query results with business interpretation per query • Risky segment count: customers with all three risk flags active

	• High-value churned customer count and avg tenure • Bonus combined query result table
CONNECTS TO	**API2:** GET /churn/summary runs these same aggregations from FastAPI **SQL6:** The risky segment logic from Query 4 becomes the v_high_risk_customers view **CP3:** Results should match CP3 pandas groupby outputs — cross-validate **ML3:** Business interpretation here informs feature importance analysis in ML3

LAB SQL5 — Create ML Feature Table

Goal: Build customer_ml_features — the formal handoff from data layer to ML layer

Prereq: SQL3 curated tables; CP4 feature definitions

▶ ACTIVITY GUIDE

SCENARIO	• The ML team needs one clean, flat table with all features ready for model training. No joins, no NULLs in key columns, correct types throughout. Build customer_ml_features — this is the formal handoff from the data/SQL layer to the ML layer.
STUDENT ACTIVITIES	74. Design the customer_ml_features table schema: all features from CP4 + original numeric columns + churn label 75. Create the table in SQL with appropriate types (INT for flags/counts, FLOAT for charges, VARCHAR for buckets) 76. Populate it using INSERT ... SELECT from fact_customer_account + dim tables, computing engineered features in SQL: 77. — service_count: SUM of 6 service binary columns 78. — high_charge_flag: CASE WHEN monthly_charges > (SELECT AVG...) THEN 1 ELSE 0 END 79. — is_long_term_customer: CASE WHEN tenure >= 24 THEN 1 ELSE 0 END 80. — auto_pay_flag: CASE WHEN payment_method LIKE "%automatic%" THEN 1 ELSE 0 END 81. Verify: SELECT COUNT(*) should be 7043; SELECT COUNT(*) WHERE monthly_charges IS NULL should be 0 82. Run a final check: SELECT churn, COUNT(*) FROM customer_ml_features GROUP BY churn — confirm class distribution
CONCEPTS	• Materialised feature table — precomputed features stored, not recalculated each time • CASE WHEN for conditional feature computation in SQL • SUM of binary columns = count of "yes" services • LIKE "%automatic%" — partial match for auto_pay detection • NULL check after insert — ML input must have no NULLs in feature columns • This table is the contract between the data layer and the ML layer
OUTPUT	• customer_ml_features table with 7043 rows, all features populated • No NULLs in any feature column (verified)

	• Churn distribution: ~5174 not churned (0), ~1869 churned (1) • Schema printed from DESCRIBE customer_ml_features
CONNECTS TO	**ML1:** ML problem definition reads column names from this table **ML2:** ML training script reads directly from customer_ml_features **API4:** GET /customer/{id}/features queries this table **DE4:** DE feature pipeline populates this same table automatically

LAB SQL6 — Create a Rule-Based High-Risk View

Goal: Pre-ML decision layer: simple SQL rules for immediate retention action

Prereq: SQL5 customer_ml_features available

▶ ACTIVITY GUIDE

SCENARIO	• Before any ML model is deployed, the retention team wants a rule-based high-risk customer list they can act on today. Build a SQL VIEW that flags customers as high-risk using simple, explainable business rules.
STUDENT ACTIVITIES	83. Define the business rules for "high risk": month-to-month contract AND tenure < 12 AND monthly_charges > dataset average 84. Write a SELECT query that returns matching customers with a risk_reason string column explaining why each was flagged 85. Test the query — verify count and inspect sample rows 86. Create the view: CREATE VIEW v_high_risk_customers AS ... 87. Query the view: SELECT * FROM v_high_risk_customers LIMIT 10 88. Count how many customers the view flags — what percentage of the total is this? 89. Discuss: what are the limitations of rule-based risk scoring vs ML-based scoring? What does ML add?
CONCEPTS	• SQL VIEW — stored query, not stored data; always reflects current table state • Subquery in WHERE: compare against a computed threshold • Risk_reason as a computed string column — useful for UI display • Rule-based vs ML-based: rules are explainable and fast to deploy; ML captures complex patterns rules miss • Views as a serving layer: API3 reads from this view directly
OUTPUT	• v_high_risk_customers view created and queryable • Flagged customer count and percentage of total printed • Sample rows showing customer_id, tenure, monthly_charges, contract, risk_reason • Written comparison: rule-based vs ML-based risk scoring (3 sentences)
CONNECTS TO	**API3:** GET /customers/high-risk reads from this exact view

RE4: High-risk customers page in React displays data from API3 which reads this view

ML2: ML model results can be compared to this rule-based list — how much do they overlap?

PHASE 3

FASTAPI BACKEND

Expose customer intelligence through REST APIs

All endpoints query the curated tables from Phase 2. Ensure telecom.db is populated before starting this phase. Run: uvicorn main:app --reload

LAB API1 — Customer Profile Endpoint

Goal: Return one customer's details by ID

Prereq: SQL3 curated tables; SQLAlchemy models

▶ ACTIVITY GUIDE

SCENARIO

- The customer support team needs to look up any customer by ID during a support call. Build GET /customer/{customer_id} — the first and most fundamental API endpoint.

STUDENT ACTIVITIES

90. Create main.py with a FastAPI app instance
91. Create a SQLAlchemy engine and session factory pointing to your database
92. Define a Pydantic CustomerResponse model with the fields to return: customer_id, tenure, contract_type, internet_service, monthly_charges, churn
93. Implement GET /customer/{customer_id}: query the customers + fact_customer_account + dim_contract tables, raise HTTPException(404) if not found
94. Return the result as the CustomerResponse model
95. Run the server: uvicorn main:app --reload
96. Test at /docs: look up a known customer ID (e.g. "7590-VHVEG")
97. Test a non-existent ID — confirm 404 is returned with a clear message

CONCEPTS

- FastAPI app setup, @app.get(), path parameter {customer_id}
- Pydantic BaseModel as response_model — automatic JSON serialisation and validation
- HTTPException(status_code=404) — standard REST pattern for missing resources
- SQLAlchemy engine + session: create_engine(), Session()
- text() for parameterised SQL — prevents SQL injection

	• FastAPI /docs auto-generated Swagger UI — test endpoints without writing a client
OUTPUT	• GET /customer/7590-VHVEG returns correct JSON with 6 fields • 404 returned with message for unknown customer ID • Swagger /docs page showing the endpoint with schema
CONNECTS TO	**SQL3:** Queries the curated tables built in SQL3 **API6:** Validation and error handling from this endpoint is standardised in API6 **RE2:** Customer search page calls this exact endpoint

LAB API2 — Churn Summary Endpoint

Goal: Expose aggregate analytics for the executive dashboard

Prereq: API1 working

▶ ACTIVITY GUIDE

SCENARIO	• The executive dashboard needs aggregate churn statistics, not individual records. Build GET /churn/summary — returning the KPIs that the leadership team checks weekly.
STUDENT ACTIVITIES	98. Add GET /churn/summary to main.py 99. Create a service layer function get_churn_summary(db) — keep business logic out of the route 100. Query: total customers, total churned, churn rate (as float 0–1) 101. Query: churn rate by contract type (list of {contract_type, churn_rate}) 102. Query: churn rate by internet service type 103. Return all of this in a single structured JSON response 104. Test at /docs — verify values match your SQL4 query results 105. Discuss: why create a service layer function rather than putting SQL directly in the route?
CONCEPTS	• Service layer pattern — separating HTTP routing from business logic • Returning nested JSON: dict containing lists of dicts • round() for clean float output in API responses • Why service layers matter: testability, reusability, cleaner route code • API contract: the JSON shape this returns is what the React dashboard expects
OUTPUT	• GET /churn/summary returning total_customers, churned, churn_rate, by_contract, by_internet_service • Values match SQL4 query results (cross-validate) • Service layer function get_churn_summary() importable separately from the route

CONNECTS TO	**SQL4:** Values must match SQL4 GROUP BY results — cross-validate **RE3:** Churn summary page calls this endpoint **API6:** get_churn_summary() gets Pydantic response model in API6

LAB API3 — High-Risk Customers Endpoint

Goal: Return rule-based flagged customers from SQL6 view

Prereq: SQL6 v_high_risk_customers view

▶ ACTIVITY GUIDE

SCENARIO	• The retention team needs an actionable list of customers to call this week. Build GET /customers/high-risk — reading from the SQL view created in SQL6 and returning a ranked list.
STUDENT ACTIVITIES	106. Add GET /customers/high-risk to main.py 107. Query the v_high_risk_customers view 108. Add optional query parameters: limit (default 50) and min_tenure / max_tenure for filtering 109. Return a list of customer objects with: customer_id, tenure, monthly_charges, contract_type, risk_reason 110. Test with default parameters — confirm results match v_high_risk_customers view 111. Test with limit=10 and max_tenure=6 — verify filtering works 112. Discuss: this is a rule-based list. How will it change when ML predictions are available (ML5)?
CONCEPTS	• Query parameters in FastAPI: def endpoint(limit: int = 50) • Query parameters vs path parameters — when to use each • Reading from a SQL VIEW — the API does not care whether it is a view or a table • Default parameter values — making endpoints usable without required inputs • Parameterised query with multiple parameters
OUTPUT	• GET /customers/high-risk returning list of flagged customers with risk_reason • Filtering by limit and max_tenure working correctly • Results consistent with v_high_risk_customers view count
CONNECTS TO	**SQL6:** Reads from the v_high_risk_customers view built in SQL6 **RE4:** High-risk customer page renders this list **ML5:** After batch scoring, this endpoint can be extended to use ML risk scores

LAB API4 — Customer Features Endpoint

Goal: Serve ML-ready feature vector per customer

Prereq: SQL5 customer_ml_features table

▶ ACTIVITY GUIDE

SCENARIO	• The ML team needs individual customer feature vectors served via API — so the prediction model can request features for any customer without direct DB access. Build GET /customer/{customer_id}/features.
STUDENT ACTIVITIES	113. Add GET /customer/{customer_id}/features to main.py 114. Query the customer_ml_features table for the given customer_id 115. Return all engineered features: service_count, tenure_bucket, high_charge_flag, is_long_term_customer, has_streaming_bundle, auto_pay_flag, monthly_charges, total_charges 116. Raise 404 if customer not found 117. Define a Pydantic CustomerFeatures response model 118. Test with a known customer ID — verify values match customer_features.csv for that customer
CONCEPTS	• Nested path: /customer/{id}/features — resource + sub-resource pattern • Pydantic response model for a feature vector • API as a feature store — decoupling ML from direct DB access • Why serve features via API vs reading DB directly in ML code: portability, caching, access control
OUTPUT	• GET /customer/7590-VHVEG/features returning all 8 feature fields • Values match customer_ml_features table row for that customer ID • 404 for unknown customer ID
CONNECTS TO	**SQL5:** Reads from customer_ml_features table built in SQL5 **ML4:** Prediction API calls this endpoint to get feature input for a customer **RE5:** Churn prediction form can pre-populate from this endpoint

LAB API5 — Prediction Stub Endpoint

Goal: API contract defined before ML model is ready

Prereq: API4 working

▶ ACTIVITY GUIDE

SCENARIO	• The React team needs the /predict-churn endpoint available now, even though the ML model is not ready yet. Build a stub that returns a realistic-looking response — ML4 will replace it with the real model.
STUDENT ACTIVITIES	119. Define a Pydantic ChurnPredictionRequest model: tenure (int), monthly_charges (float), contract_type (str), service_count (int) 120. Add POST /predict-churn to main.py 121. Return a stub response: {"customer_id": input or "N/A", "risk_score": 0.78, "prediction": "Likely to churn", "note": "stub — real model in ML4"} 122. Add a "note" field making it clear this is a stub 123. Test via /docs: submit a JSON body and verify the response 124. Verify that submitting wrong types (e.g. tenure as a string) returns a 422 Validation Error automatically from Pydantic
CONCEPTS	• POST vs GET: POST when sending a request body; GET for retrieval • Pydantic request body validation — free type checking and 422 errors • Stub pattern: defining the API contract before implementing the logic • 422 Unprocessable Entity — Pydantic returns this automatically for type errors • API-first development: backend and React can both proceed before ML is ready
OUTPUT	• POST /predict-churn accepting JSON body and returning stub risk_score and prediction • 422 error verified for invalid input types • Note field confirming this is a stub
CONNECTS TO	**ML4:** Real model replaces this stub in ML4 **RE5:** Churn prediction page posts to this endpoint **API6:** Pydantic request model is enhanced with validators in API6

LAB API6 — Add Validation, Logging & Auth

Goal: Production discipline: field constraints, structured logging, token-based auth

Prereq: API1–5 all working

▶ ACTIVITY GUIDE

SCENARIO	• Raw working code is not production code. Add the discipline that real APIs need: input validation with field constraints, structured logging, consistent error response format, and HTTP status codes. Harden all 5 existing endpoints.
STUDENT ACTIVITIES	125. Add field-level Pydantic validators to ChurnPredictionRequest: tenure must be >= 0 and <= 100; monthly_charges must be > 0 126. Add a consistent error response model: ErrorResponse(detail: str, status_code: int) 127. Wrap all DB queries in try/except — return 500 with ErrorResponse if DB is unavailable 128. Add Python logging to every route: log the incoming request (method, path, key params) and the response status 129. Add a 404 handler for all routes using @app.exception_handler(404) 130. Test: valid request, invalid tenure (negative), unknown customer ID, simulate DB down 131. Add a basic token check: read an API_KEY from environment variable; any request without matching X-API-Key header gets 401 132. Discuss: how does this differ from full OAuth2/JWT? What would change in production?
CONCEPTS	• Pydantic @validator decorator — custom field-level validation • Consistent error response format — all errors return the same shape • Exception handling in FastAPI: try/except inside routes + @app.exception_handler • Request logging: log method, path, params, response code • API key auth: os.getenv("API_KEY"), Header() parameter, 401 Unauthorized • Token-based auth overview: X-API-Key header check is a simple version of what JWT does
OUTPUT	• All 5 endpoints return structured error responses (not raw FastAPI defaults) • tenure=-1 returns 422 with custom validation message • Unknown customer_id returns 404 with ErrorResponse shape • All requests logged with timestamp, path, status • Request without valid API key returns 401
CONNECTS TO	**RE1:** React must handle 401, 404, 422 status codes gracefully **ML4:** Prediction endpoint validation ensures the model never receives out-of-range features **TOC:** Satisfies TOC requirement: basic auth concept (token-based overview)

PHASE 4

REACT (API CONSUMER)

Customer retention dashboard — UI as API consumer

React scope is intentionally limited. Goal: students understand how a UI talks to a backend. Full frontend engineering is not the objective.

LAB RE1 — Setup React App

Goal: Create project, configure API base URL, test one live call

Prereq: FastAPI running on localhost:8000

▶ ACTIVITY GUIDE

SCENARIO	• Set up the React project that will become the Customer Retention Dashboard. Configure the API base URL and test one live API call before building any pages.
STUDENT ACTIVITIES	133. Run npx create-react-app customer-dashboard 134. Install axios: npm install axios 135. Create src/api.js: export BASE_URL reading from REACT_APP_API_URL env variable or defaulting to http://localhost:8000 136. Add FastAPI CORS middleware: allow origins=["http://localhost:3000"] in main.py 137. In App.js, add a useEffect that calls GET /churn/summary on load and console.log the result 138. Start both servers: uvicorn main:app --reload and npm start 139. Open browser console — confirm the API response is visible without CORS errors 140. Handle the API call properly: show "Loading..." before data arrives, show error message if call fails
CONCEPTS	• create-react-app project structure: src/, public/, package.json • CORS: why the browser blocks cross-origin requests and how CORS middleware allows it • Environment variables in React: REACT_APP_ prefix required • axios vs fetch: axios has cleaner error handling and automatic JSON parsing • useEffect with [] dependency — runs once on component mount • Loading/error/data state pattern — the foundation of every React data-fetch component
OUTPUT	• React dev server at localhost:3000 • API response visible in browser console (no CORS errors) • Loading and error states rendering correctly
CONNECTS TO	**RE2:** Same BASE_URL pattern used in every subsequent page **API6:** If API key auth is enabled in API6, React must include the X-API-Key header here

LAB RE2 — Customer Search Page

Goal: User enters customer ID → fetches and displays profile card

Prereq: RE1 setup complete; API1 working

▶ ACTIVITY GUIDE

SCENARIO	• Build the Customer Search page. A support agent enters a customer ID and gets their full profile — including tenure, contract, charges, and churn status.
STUDENT ACTIVITIES	141. Create src/pages/CustomerSearch.js 142. Add a controlled text input for customer ID (useState for the input value) 143. Add a Search button — onClick triggers the API call 144. Call GET /customer/{id} with the input value using axios 145. Show a loading spinner/message while the request is in flight 146. Display the result in a card layout: customer_id, tenure, contract_type, internet_service, monthly_charges, churn status 147. Handle 404 gracefully: show "Customer not found" instead of a crash 148. Show the churn field with colour coding: red background for churned, green for active 149. Import and add CustomerSearch to App.js with a simple tab or link
CONCEPTS	• Controlled component: input value tied to state via value + onChange • onClick-triggered async fetch (not useEffect — user must click) • e.response?.status — optional chaining to safely read error status code • Handling HTTP errors: axios throws on 4xx/5xx; check status in catch block • Conditional styling based on data: background colour from churn field • Null guard: customer && ... — only render result when data exists
OUTPUT	• Customer search working: enter ID, click Search, see profile card • "Customer not found" shown for unknown IDs (not a crash) • Churn status colour-coded (red/green)
CONNECTS TO	**API1:** GET /customer/{id} is the endpoint consumed here **RE3:** Same loading/error/data pattern used in every subsequent page

LAB RE3 — Churn Summary Page

Goal: Display executive KPIs on load — no user input required

Prereq: RE2; API2 working

▶ **ACTIVITY GUIDE**

SCENARIO	• Build the Churn Analytics page. On load it fetches /churn/summary and displays the key metrics the executive team tracks — total customers, churn rate, and a breakdown by contract type.
STUDENT ACTIVITIES	150. Create src/pages/ChurnSummary.js 151. On mount (useEffect with []), fetch GET /churn/summary 152. Display three metric cards: total customers, total churned, churn rate (as a percentage) 153. Display a contract-type churn table: contract_type | churn_rate columns 154. Format churn_rate as a percentage: (rate * 100).toFixed(1) + "%" 155. Add a simple visual: for each contract type, show a coloured bar whose width is proportional to its churn rate 156. Handle loading and error states 157. Add a page title and a one-sentence description of what the page shows
CONCEPTS	• useEffect auto-fetch on mount — no user trigger needed • Array rendering with .map() and key prop • toFixed(1) — format floats to 1 decimal place for display • Proportional bar: width = rate * 100 * scale_factor pixels • Destructuring: const {total_customers, churned, churn_rate, by_contract} = data
☑ **OUTPUT**	• Three metric cards showing correct values from API • Contract-type churn table with percentage formatting • Proportional bars showing visual comparison
🔗 **CONNECTS TO**	**API2:** GET /churn/summary is the endpoint consumed here **RE4:** Same table rendering pattern used for high-risk customer list

LAB RE4 — High-Risk Customers Page

Goal: Render paginated list from /customers/high-risk

Prereq: RE3; API3 working

▶ **ACTIVITY GUIDE**

SCENARIO	• Build the High-Risk Customers page. The retention team uses this list to prioritise their outreach calls. Render the /customers/high-risk list as an interactive table with sorting and a count summary.
STUDENT ACTIVITIES	158. Create src/pages/HighRiskCustomers.js 159. On mount, fetch GET /customers/high-risk (default limit 50) 160. Render results in an HTML table: customer_id | tenure | monthly_charges | contract_type | risk_reason 161. Show a count banner: "X high-risk customers identified" 162. Add a "Load More" button that refetches with limit increased by 50 163. Add a sort toggle on the tenure column: clicking the header sorts ascending/descending 164. Format monthly_charges as currency: "$" + value.toFixed(2) 165. Handle the case where 0 results are returned: show "No high-risk customers found"
CONCEPTS	• useEffect with dependency [limit]: re-fetches when limit state changes • Spread to sort without mutating: [...array].sort() • Toggle state: const [sortAsc, setSortAsc] = useState(true) • Query parameters in axios: template literal `?limit=${limit}` or params: {limit} • List rendering with .map(), key={customer.customer_id}
OUTPUT	• High-risk customer table rendering correctly with all 5 columns • Load More increases the displayed count correctly • Tenure column sort toggling between ascending and descending • "No high-risk customers found" message for empty response
CONNECTS TO	**API3:** GET /customers/high-risk is the endpoint consumed here **ML5:** After batch scoring, this page can be extended to show ML risk scores alongside rule-based flags

LAB RE5 — Churn Prediction Page

Goal: Form → POST → risk score display — clearest full-stack integration

Prereq: RE4; API5 stub or ML4 real model

▶ ACTIVITY GUIDE

SCENARIO	• Build the Churn Prediction page. A retention agent enters customer details and gets an instant churn risk score. This is the most complete UI-to-API-to-ML integration in the entire course.

STUDENT ACTIVITIES	166. Create src/pages/ChurnPrediction.js 167. Build a form with 4 controlled inputs: tenure (number), monthly_charges (number), contract_type (select: Month-to-month / One year / Two year), service_count (number 0–6) 168. On submit, POST to /predict-churn with the form values as a JSON body 169. Display the result: risk_score as a percentage, prediction text, confidence level 170. Add visual risk indicator: green for low risk (<0.3), amber for medium (0.3–0.6), red for high (>0.6) 171. Show a loading state on the submit button while the request is in flight 172. Handle validation errors (422): display the Pydantic error message to the user 173. Test with stub response first; retest after ML4 wires up the real model 174. Full end-to-end test: enter values → POST → model response → coloured risk display
CONCEPTS	• POST with JSON body: axios.post(url, data) • Controlled select element: value={formData.contract_type} onChange={...} • try/catch/finally: loading=false always runs in finally • Handling 422 validation errors from Pydantic: e.response.data.detail • Conditional styling: ternary operator for three-level colour coding • Full-stack integration test: React → FastAPI → ML model → React
OUTPUT	• Prediction form submitting correctly with all 4 fields • Risk score displayed with colour indicator (green/amber/red) • 422 error message shown for invalid tenure input • Real model predictions shown after ML4 is complete
CONNECTS TO	**API5:** POST /predict-churn is the endpoint consumed here **ML4:** After ML4, stub is replaced — this page shows real model predictions **Full stack:** This completes the React → FastAPI → ML → React circle

PHASE 5

DATA ENGINEERING

Batch customer intelligence pipeline — with PySpark

This dataset suits batch DE: one daily customer extract, not a real-time event stream. The pipeline is: raw CSV → staging → cleaned → features → PySpark analytics → ML scoring → API. Airflow orchestrates all stages.

TOC LINK	PySpark is placed here (DE5) because it belongs in the DE track, not as a bonus at the end. Students meet PySpark after they already understand the data well through pandas (CP3) and the feature pipeline (DE4). The placement makes the contrast explicit: same data, same aggregations, different engine — that IS the lesson.

Full PySpark depth (multi-file, performance tuning, production ETL) is in the Network Dataset lab track.

LAB DE1 — Design Customer Data Pipeline Architecture

Goal: Map every layer before writing code

Prereq: CP5 pipeline and SQL schema complete

▶ ACTIVITY GUIDE

SCENARIO	• Before building anything, design the complete customer data pipeline on paper. Map every layer from raw CRM extract to ML-ready feature table and API/dashboard.
STUDENT ACTIVITIES	175. Draw the pipeline layers: CRM Extract → Raw File → Staging Table → Cleaned Table → Feature Table → ML Scoring → Dashboard/API 176. For each layer, define: what data format, what tool handles it, who reads it next 177. Identify the two key quality gates: schema validation (before staging load) and data quality check (before feature build) 178. Define the incremental processing question: what happens when the same customer appears in two consecutive daily extracts? 179. Map each layer to the labs that implement it: CP5 handles layers 1–3, DE2 formalises ingestion, DE3 formalises cleaning, etc. 180. Produce a one-page architecture document: layer diagram + tool mapping + quality gate definitions
CONCEPTS	• Data pipeline layers: ingestion, staging, cleaning, feature engineering, serving • Source of truth: each layer has exactly one upstream source and one defined format • Quality gates: validation checkpoints that stop bad data from advancing downstream • Idempotency: running the pipeline twice on the same input should produce the same output • Batch vs incremental: full reload each time vs processing only new/changed records
OUTPUT	• Architecture diagram: 7 layers with tool/format annotations • Quality gate definitions: what is checked, what happens on failure • Layer-to-lab mapping table • Incremental strategy decision: student-written upsert logic description
CONNECTS TO	**DE2:** Implements the ingestion layer defined here **DE3:** Implements the cleaning layer **DE4:** Implements the feature engineering layer **DE7:** Connects all layers in the Airflow orchestration DAG

LAB DE2 — Build the Ingestion Flow

Goal: Schema-validate daily extract and load to staging

Prereq: DE1 architecture designed

▶ ACTIVITY GUIDE

SCENARIO

- Simulate the daily arrival of a customer extract. Validate it, load it into the staging table, and log the result.
- This is the first step every data pipeline must handle correctly.

STUDENT ACTIVITIES

181. Create the folder structure: data/landing/, data/raw/, data/rejected/, data/logs/
182. Place customer_churn.csv into data/landing/
183. Write detect_files() — scans landing/ and returns list of CSV files
184. Write validate_schema(filepath) — reads the header row and checks it matches EXPECTED_COLS (21 column names exactly)
185. Write load_to_staging(filepath, engine) — loads valid CSV into stg_customer_raw, returns row count
186. Write log_ingestion(filename, status, rows, reason) — writes to ingestion_log table
187. Write process_landing() — orchestrates: detect → validate → load → log for each file
188. Test: run with a valid file (expect LOADED), an invalid file missing a column (expect REJECTED)
189. Verify: SELECT * FROM ingestion_log shows both entries

```
EXPECTED_COLS =
["customerID","gender","SeniorCitizen","Partner","Dependents","tenure","PhoneService",
"MultipleLines","InternetService","OnlineSecurity","OnlineBackup","DeviceProtection",
"TechSupport","StreamingTV","StreamingMovies","Contract","PaperlessBilling","PaymentMethod",
"MonthlyCharges","TotalCharges","Churn"]
def validate_schema(filepath):
    with open(filepath) as f: header = next(csv.reader(f))
    return header == EXPECTED_COLS, header
def load_to_staging(filepath, engine):
    df = pd.read_csv(filepath)
    df.to_sql("stg_customer_raw", engine, if_exists="replace", index=False)
    return len(df)
```

CONCEPTS

- Schema validation: exact column name and order check
- Data zones: landing (unprocessed) vs raw (validated+loaded) vs rejected (failed)
- to_sql(if_exists="replace") — full reload of staging each time
- ingestion_log table: filename, status, row_count, reason, loaded_at
- Atomic processing: validate fully before loading — never partially load

OUTPUT

- Valid file: LOADED status in ingestion_log, 7043 rows in stg_customer_raw
- Invalid file (missing column): REJECTED status in log with column mismatch message
- Process can be re-run without duplication (replace replaces the staging table)

CONNECTS TO

SQL2: Manual staging load from SQL2 is automated here
DE3: DE3 reads from stg_customer_raw after this step succeeds
DE7: Ingestion step becomes the first Airflow task

LAB DE3 — Build the Cleaning Stage

Goal: Reuse CustomerCleaner to transform staging into trusted data

Prereq: DE2 staging loaded; CP2 CustomerCleaner available

▶ ACTIVITY GUIDE

SCENARIO	• The staging table has raw, messy data. Build the cleaning stage that transforms it into the trusted customers and fact_customer_account tables — reusing CustomerCleaner from Phase 1.
STUDENT ACTIVITIES	190. Write clean_staging(engine) — reads stg_customer_raw into a DataFrame, runs CustomerCleaner.clean(), writes to a cleaned_customers table 191. Write build_curated_tables(engine) — populates dim_contract, dim_payment, customers, fact_customer_account from the cleaned staging data 192. Add a row count check: assert cleaned_customers row count == stg_customer_raw row count (no rows dropped unless intentional) 193. Add data quality assertions: no NULLs in customer_id or monthly_charges; churn column values are only 0 or 1 194. Write a simple quality report: for each check, print PASS or FAIL with count 195. Test: run clean_staging() followed by build_curated_tables() — verify all four curated tables are populated
CONCEPTS	• Reusing CustomerCleaner — the class built in CP2 earns its keep here • Read from staging → clean → write to curated: the ETL clean stage • assert for data contracts: fail fast if quality check fails rather than silently passing bad data • Quality report: PASS/FAIL per check — visible audit trail for the pipeline • if_exists="replace" for cleaning reruns — cleaning is always a full reload
OUTPUT	• cleaned_customers table populated with 7043 rows, clean column names, correct types • Quality report: all checks PASS • dim_contract, dim_payment, customers, fact_customer_account all populated • Row count assertion passing (no accidental row drops)
CONNECTS TO	**CP2:** CustomerCleaner class is imported and reused here directly **DE4:** Feature build stage reads from cleaned_customers **DE7:** This becomes the second Airflow task in the full pipeline

LAB DE4 — Build the Feature Pipeline

Goal: Automatically generate customer_ml_features from cleaned data

Prereq: DE3 cleaned_customers table

▶ ACTIVITY GUIDE

SCENARIO	• The ML team needs consistently engineered features regenerated every time new customer data arrives. Build the feature pipeline stage that populates customer_ml_features automatically from the cleaned data.
STUDENT ACTIVITIES	196. Write build_features(engine) — reads cleaned_customers, computes all 6 features from CP4, writes to customer_ml_features table 197. Ensure the feature logic exactly matches CP4: same thresholds (median for high_charge_flag), same column names, same data types 198. Add a validation step: count of customer_ml_features should equal count of cleaned_customers 199. Add a schema check: confirm all 6 feature columns are present and non-null 200. Test: run the function and query customer_ml_features — spot-check 3 customers against the CSV output from CP4 201. Discuss: if the CP4 feature definitions change, what do you need to update in the pipeline?
CONCEPTS	• Feature pipeline: applying engineered feature logic consistently on every pipeline run • Consistency: thresholds computed on the current batch (median) not hardcoded — this is correct • DE is what makes ML reproducible: same pipeline = same features = comparable model runs • Schema as a contract: if CP4 adds a feature, DE4 must be updated to match
OUTPUT	• customer_ml_features populated with 7043 rows, all 6 features present • Schema check passing: no NULLs in any feature column • Spot-check: 3 sample customers in customer_ml_features match values from CP4 CSV output
CONNECTS TO	**CP4:** Feature logic must exactly match the definitions in CP4 **SQL5:** This Python-based feature table is the automated equivalent of the SQL5 table **ML2:** ML training reads from customer_ml_features which this step populates **DE7:** This becomes the third Airflow task

LAB DE5 — Process Customer Data with PySpark

Goal: Apply PySpark to the same data students know from pandas — same results, distributed engine

Prereq: DE4 feature pipeline complete; CP3 pandas churn rates known for cross-validation

▶ ACTIVITY GUIDE

SCENARIO	• You have built the feature pipeline in pandas (DE4). Now ask: what if this were millions of customers, not 7,000? Process the same customer data with PySpark — replicate the churn aggregations you already know, see the same results, and understand why the engine matters at scale.

STUDENT ACTIVITIES	202. Create a SparkSession: local[2], 4g driver memory (7K rows is tiny — this config is intentionally minimal) 203. Define a manual schema (StructType) for the CSV — all correct types including StringType for TotalCharges 204. Read customer_churn.csv with the defined schema — print printSchema() and compare to pandas dtypes 205. Fix TotalCharges blanks: use F.regexp_replace() to replace blank strings, then cast to DoubleType 206. Encode the Churn column: use F.when(F.col("Churn")=="Yes", 1).otherwise(0) — Spark equivalent of map() 207. GroupBy Contract: compute COUNT(*) as customer_count and AVG(churn_encoded) as churn_rate — show results 208. GroupBy InternetService: compute AVG(MonthlyCharges) and AVG(churn_encoded) — show results 209. Cross-validate: Spark churn rates must exactly match your CP3 pandas groupby results 210. Write the contract-level summary as Parquet: data/spark_output/contract_summary/ partitioned by Contract 211. Read the Parquet back, print schema and count — confirm round-trip is clean
CONCEPTS	• SparkSession: master("local[2]") uses 2 cores; local[*] would use all and is unnecessary here • Manual schema vs inferSchema: inference reads the whole file to guess types — always wrong for TotalCharges • F.regexp_replace() — Spark equivalent of str.replace(); cleans blank strings before casting • F.when().otherwise() — Spark equivalent of np.where() or pd.Series.map() • groupBy().agg() with F.count() and F.avg() — same logic as pandas, but distributed execution • partitionBy() on write — same Parquet partitioning pattern used in the network dataset DE labs • Key insight: same results as pandas, different engine — the skill transfers to any dataset size
OUTPUT	• SparkSession running without OutOfMemoryError on 16 GB laptop • Contract churn rates printed — must exactly match CP3 pandas results • Parquet written to data/spark_output/contract_summary/ with partition folders • Round-trip read: schema and row count confirmed
CONNECTS TO	**CP3:** Spark churn rates must match CP3 pandas results exactly — cross-validate **DE6:** Incremental processing concept applies to Spark pipelines too at scale **DE8:** This Spark job can be added as an optional task in the Airflow orchestration DAG **Network Labs:** Full PySpark depth (SP1–SP7) is in the network dataset track — this is the conceptual bridge **TOC:** Satisfies: PySpark Fundamentals + Data Processing with PySpark modules

LAB DE6 — Design Incremental Processing

Goal: Handle new, updated, and unchanged customers — prevent duplicates

Prereq: DE5 PySpark lab complete; DE4 feature pipeline working

▶ ACTIVITY GUIDE

SCENARIO	• Daily customer extracts will contain a mix of new customers, updated existing customers, and unchanged records. Design and implement incremental processing so re-running the pipeline on the same data does not create duplicates.
STUDENT ACTIVITIES	212. Discuss the three cases: new customer (INSERT), updated customer (UPDATE), unchanged (skip) 213. Implement upsert logic for the customers table using SQLAlchemy: INSERT ... ON CONFLICT (customer_id) DO UPDATE 214. Test Case 1 — new customer: add one row to the CSV with a new customer_id, re-run pipeline, verify count increases by 1 215. Test Case 2 — updated customer: change tenure of an existing customer_id in the CSV, re-run, verify the row is updated not duplicated 216. Test Case 3 — same file re-run: re-run with the original CSV, verify count stays at 7043 with no duplicates 217. Add a pipeline run log: record file_name, rows_inserted, rows_updated, rows_unchanged, run_timestamp 218. Discuss: why is idempotency important in production pipelines?
CONCEPTS	• Upsert pattern: INSERT ... ON CONFLICT DO UPDATE — one statement handles both cases • Idempotency: running the pipeline twice on the same input produces the same result • Incremental vs full reload: full reload is simpler but slower; incremental is faster but more complex • Pipeline run log: counting inserts vs updates vs unchanged gives visibility into data drift • EXCLUDED: in upsert SQL, EXCLUDED.col refers to the new incoming value
OUTPUT	• Test Case 1: count increases by 1 after adding new customer • Test Case 2: tenure updated, row count unchanged • Test Case 3: identical re-run produces identical table (0 new rows) • Pipeline run log entry for each test run
CONNECTS TO	**DE6:** Idempotency is a prerequisite for reliable data quality checks **DE7:** Incremental processing is specified in the Airflow DAG task for customer loading

LAB DE7 — Add Data Quality Checks

Goal: Programmatic quality gates — stop bad data before it reaches ML

Prereq: DE5 and DE6 complete

▶ ACTIVITY GUIDE

SCENARIO	• Data quality is not optional. Build a quality check layer that runs after every pipeline stage and generates a quality report — before any data reaches the ML model or the API.
STUDENT ACTIVITIES	219. Define 6 quality checks as functions: check_null_rate(df, col, threshold), check_value_range(df, col, min, max), check_allowed_values(df, col, allowed_set), check_row_count(df, expected_min), check_no_duplicates(df, key_col), check_churn_distribution(df) 220. Run null rate check: monthly_charges and tenure must have 0% nulls 221. Run value range check: tenure must be 0–100; monthly_charges must be > 0 222. Run allowed values check: contract must be one of {Month-to-month, One year, Two year} 223. Run row count check: must have at least 7000 rows (catches truncated files) 224. Run duplicate check: no duplicate customer_id values 225. Produce a quality_report.json with: check name, PASS/FAIL, value found, threshold 226. If any CRITICAL check fails: raise an exception to stop the pipeline 227. If WARNING checks fail: log warning but continue
CONCEPTS	• Data quality as code: programmatic checks, not manual inspection • CRITICAL vs WARNING severity: not all failures should stop the pipeline • Quality report as an artefact: saved as JSON for audit and trend tracking • ~df[col].isin(allowed) — finding rows with unexpected category values • Threshold-based checks: 0% nulls for critical columns; perhaps 1% for less critical
OUTPUT	• quality_report.json with 6 checks, each showing PASS/FAIL, value, and threshold • Pipeline stops with clear error message when a CRITICAL check fails • WARNING checks logged but pipeline continues
CONNECTS TO	**ML2:** Quality-checked customer_ml_features is the safe input to model training **DE7:** Quality check becomes a gate task in the Airflow DAG — pipeline only continues if checks pass

LAB DE8 — Design Orchestration Flow

Goal: Automate the full pipeline — DE1 through DE7 — with Airflow

Prereq: DE1–DE7 all working as standalone functions

▶ **ACTIVITY GUIDE**

SCENARIO	• All pipeline stages are built. Connect them into a single automated daily DAG in Airflow so the whole pipeline runs with one trigger — from raw file arrival to ML-ready features.
STUDENT ACTIVITIES	228. Map all pipeline stages to Airflow tasks: ingest, validate_quality, clean, build_features, run_upsert, notify 229. Create dags/customer_pipeline_dag.py with a @daily scheduled DAG 230. Define each stage as a PythonOperator calling the relevant function 231. Chain: ingest >> validate_quality >> clean >> build_features >> run_upsert >> notify 232. Add a quality gate: if validate_quality task fails, all downstream tasks are skipped (Airflow handles this automatically via task chaining) 233. Add a simple notify() task that logs "Customer intelligence pipeline complete — X customers processed" 234. Trigger the DAG manually and watch all 6 tasks succeed in the Airflow UI 235. Break one task deliberately (e.g. delete the landing file) — confirm downstream tasks do not run 236. Discuss: how would you extend this DAG to trigger ML batch scoring (ML5) after build_features completes?
CONCEPTS	• Airflow DAG: directed acyclic graph of dependent tasks • PythonOperator: calls a Python function as an Airflow task • Task chaining with >>: downstream only runs if upstream succeeds • schedule_interval="@daily": runs once per day • catchup=False: do not backfill missed runs • Quality gate as a task: validate_quality failure propagates as a block to all downstream
OUTPUT	• Airflow DAG with 6 tasks showing green Success on a full run • Task failure propagation confirmed (break ingest → validate and clean do not run) • Notify task log visible: "X customers processed" • Written answer: how to add ML batch scoring as task 7
CONNECTS TO	**ML5:** Batch scoring task (ML5) is the natural 7th task in this DAG **API:** APIs serve the data this pipeline populates — pipeline is what keeps APIs fresh

PHASE 6

MACHINE LEARNING

Build a customer churn prediction system

This dataset has a pre-existing Churn column — no label engineering required. The target is clear. Focus on correct problem setup, meaningful evaluation, and model interpretability.

LAB ML1 — Define the ML Problem

Goal: Identify target, features, drops, and encoding needs — the most important step

Prereq: SQL5 customer_ml_features; CP4 understanding

▶ ACTIVITY GUIDE

SCENARIO

- Before touching sklearn, define the ML problem correctly. Identify the target, the features, what to drop, and what encoding is still needed. Getting this step right determines whether the model is useful.

STUDENT ACTIVITIES

237. Load customer_ml_features (from SQL5 or CP4 CSV)
238. Identify the target column: churn (already encoded as 0/1)
239. Identify the drop column: customer_id (identifier, not a feature)
240. List all numeric columns: tenure, monthly_charges, total_charges, service_count, high_charge_flag, is_long_term_customer, auto_pay_flag, has_streaming_bundle
241. List the remaining categorical columns: contract_type, internet_service — these still need encoding
242. Apply one-hot encoding to contract_type and internet_service using pd.get_dummies()
243. Print the final feature set X and target y — print shapes
244. Print the class balance: y.value_counts(normalize=True) — note the ~73%/27% split
245. Discuss: is this imbalance a problem? What metric should we NOT use as primary? (accuracy — predict all 0 = 73% accuracy trivially)

CONCEPTS

- Feature matrix X and target vector y — the two inputs every sklearn model needs
- Drop identifier columns: customer_id carries no signal, causes data leakage if kept
- One-hot encoding: pd.get_dummies() creates binary columns for each category level
- drop_first=False: keep all levels for interpretability (drop_first avoids multicollinearity but makes coefficients harder to read)
- Class imbalance: ~73/27 split — mild; note for evaluation
- Primary metric should be F1, precision, or recall for the churned class — not overall accuracy

OUTPUT

- Feature matrix X with correct columns (customer_id and churn removed)
- One-hot encoded contract_type and internet_service columns visible
- X.shape: (7043, ~20 columns), y.shape: (7043,)
- Class balance printed: ~0.735 / 0.265

CONNECTS TO

ML2: This X and y are the direct inputs to model training
SQL5: customer_ml_features table is the data source for this problem definition
CP4: Feature columns defined in CP4 are the foundation of X here

LAB ML2 — Build and Compare Baseline Models

Goal: Train Logistic Regression and Decision Tree — compare on F1 not accuracy

Prereq: ML1 X and y defined

▶ ACTIVITY GUIDE

SCENARIO	• Train two classifier models and evaluate them correctly. Compare Logistic Regression (simple, interpretable) and Decision Tree (non-linear, rule-based) to understand their trade-offs on this churn dataset.
STUDENT ACTIVITIES	246. Split X and y: train_test_split(test_size=0.2, stratify=y, random_state=42) 247. Train a LogisticRegression(max_iter=1000) and a DecisionTreeClassifier(max_depth=5) 248. Predict on the test set for both models 249. For each model: print classification_report with target_names=["Active","Churned"] 250. For each model: print confusion_matrix 251. Compute 5-fold cross-validation F1 score for both: cross_val_score(scoring="f1") 252. Create a comparison table: Model | Accuracy | Precision (Churned) | Recall (Churned) | F1 (Churned) | CV F1 253. Choose the better model based on F1 for the Churned class and justify the choice 254. Save both models with joblib: models/logistic_churn.pkl and models/tree_churn.pkl
CONCEPTS	• stratify=y: ensures both train and test have same class ratio (~73/27) • max_iter=1000: LogisticRegression needs more iterations to converge on this dataset • max_depth=5: prevents Decision Tree overfitting on training data • classification_report: per-class precision, recall, F1 — read the "Churned" row • Confusion matrix: TP=correctly predicted churners, FN=missed churners (costly) • Cross-validation: 5-fold CV gives more reliable F1 than a single split • Model selection: higher recall for Churned class = fewer missed churners = better for retention
OUTPUT	• classification_report for both models • Confusion matrices for both models • CV F1 scores: both models (Logistic typically ~0.56–0.60; Decision Tree ~0.55–0.62) • Comparison table with 5 metrics per model • Written justification for model choice • Both .pkl files saved in models/
CONNECTS TO	**ML3:** Chosen model is inspected for feature importance/coefficients in ML3 **ML4:** Best model is loaded in the prediction API in ML4 **API5:** This model replaces the stub in /predict-churn

LAB ML3 — Interpret Model Behaviour

Goal: Turn model output into business language

Prereq: ML2 models saved

▶ ACTIVITY GUIDE

SCENARIO	• A model that cannot be explained cannot be trusted by a business. Interpret the model — which features drive churn the most, which customers look most at risk, and what this means in plain English for the retention manager.
STUDENT ACTIVITIES	255. Load the best model from ML2 (Logistic Regression: inspect coefficients; Decision Tree: inspect feature importances) 256. For Logistic Regression: extract coef_ as a Series with feature names; sort by absolute value; plot top 10 257. For Decision Tree: extract feature_importances_ similarly 258. Identify the top 3 features most associated with higher churn probability 259. Score all customers in the test set: predict_proba() to get churn probability 260. Find the top 20 highest-risk customers in the test set — print their customer IDs and risk scores 261. Check: do the top-risk customers match the rule-based high-risk list from SQL6? (partial overlap expected) 262. Write a 5-sentence business interpretation: "Customers most likely to churn are those who..." targeted at a retention manager
CONCEPTS	• coef_: Logistic Regression coefficients — positive = more churn, negative = less churn • feature_importances_: Decision Tree — how much each feature reduces impurity • predict_proba()[:,1]: probability of the positive class (churn=1) • Absolute value of coefficients: the sign tells direction, the magnitude tells importance • Business translation: turn numerical findings into plain language for non-technical stakeholders • Expected findings: month-to-month contract, short tenure, electronic check payment → highest churn drivers
OUTPUT	• Top 10 feature importance chart (Logistic: coefficients, Tree: importances) • Top 20 highest-risk customers with risk scores • Overlap analysis: how many of the top-20 ML customers appear in SQL6 rule-based list • Written 5-sentence business interpretation
CONNECTS TO	**SQL6:** Compare ML top-risk vs rule-based high-risk — quantify overlap **ML4:** Feature importance findings inform which features are most important in the prediction API

RE3: Business interpretation is what the Churn Summary page communicates visually

LAB ML4 — Replace Stub API with Real Model

Goal: Wire trained model into /predict-churn — complete the full stack

Prereq: ML2 models saved; API5 stub running

▶ ACTIVITY GUIDE

SCENARIO

- Replace the stub in /predict-churn with the real trained model. The complete flow is: JSON request → preprocess → model inference → JSON response. Test end-to-end.

STUDENT ACTIVITIES

263. Create predict.py: load the best model from models/ at module level
264. Define preprocess_input(tenure, monthly_charges, contract_type, service_count) — creates a properly encoded single-row DataFrame matching the training feature columns
265. Define predict_churn(tenure, monthly_charges, contract_type, service_count) — calls preprocess_input, then model.predict() and predict_proba()
266. Return: {"risk_score": float, "prediction": "Likely to churn" / "Unlikely to churn", "confidence": float}
267. In main.py: import predict_churn from predict.py; replace the stub with a real call
268. Test via /docs: submit a month-to-month customer with short tenure → expect high risk
269. Test: submit a two-year contract customer with long tenure → expect low risk
270. Full end-to-end: test from RE5 React form → API → model → coloured response in UI

CONCEPTS

- Module-level model load: load once at import time, not on every request
- Feature encoding at inference time must exactly match training encoding
- Manual one-hot encoding for inference: set the right contract_type_ column to 1, rest to 0
- predict_proba()[:,1]: index 1 = probability of positive class (churn)
- End-to-end test: the real test is the full React → API → model → React flow

OUTPUT

- POST /predict-churn returning real risk_score and prediction (not stub)
- Month-to-month short tenure: high risk score (>0.6)
- Two-year long tenure: low risk score (<0.3)
- Full-stack test: RE5 form showing coloured real predictions

CONNECTS TO

API5: Replaces the stub built in API5
RE5: Full end-to-end test with the React prediction form
ML5: This predict_churn() function is used in the batch scoring job

LAB ML5 — Build a Batch Scoring Job

Goal: Score all customers daily — output feeds API, React, and future AI

Prereq: ML4 predict.py complete

▶ ACTIVITY GUIDE

SCENARIO	• The model can predict one customer at a time. Now score all 7043 customers in batch — producing a customer_risk_table that the dashboard, the retention team, and the AI assistant can all consume.
STUDENT ACTIVITIES	271. Create batch_score.py 272. Load all customers from customer_ml_features 273. Apply predict_churn() to every row using df.apply() 274. Extract risk_score, prediction, confidence into new columns 275. Add a scoring_date column: datetime.today().strftime("%Y-%m-%d") 276. Save to customer_risk_table (SQL table or CSV): customer_id, risk_score, prediction, confidence, scoring_date 277. Print a summary: count of "Likely to churn" vs "Unlikely to churn" — verify ~27% churn rate aligns with training distribution 278. Print top 10 highest risk_score customers — cross-check against SQL6 view (do the rule-based high-risk customers have high ML scores?) 279. Discuss: how would you add this batch scoring as the 7th task in the DE7 Airflow DAG?
CONCEPTS	• Batch inference: applying a prediction function row-by-row to all records • df.apply(axis=1): calls function for each row; slower than vectorised but correct here • scoring_date column: essential for time-series risk tracking • customer_risk_table: the central output that feeds React dashboard, API, and future AI assistant • Distribution check: predicted churn rate should be close to actual churn rate (~26.5%)
OUTPUT	• customer_risk_table with 7043 rows: customer_id, risk_score, prediction, confidence, scoring_date • Predicted churn distribution: close to ~27% "Likely to churn" • Top 10 risk score customers printed and cross-checked with SQL6 view • Written answer: how to add batch scoring as DAG task 7
CONNECTS TO	**DE7:** This batch scoring job is the natural 7th task in the Airflow DAG **API3:** High-risk customers endpoint can be extended to join with customer_risk_table ML scores **RE4:** High-risk customers page can display ML risk scores alongside rule-based flags **Future AI:** customer_risk_table is the data source for the AI assistant churn prediction feature